

DevFreela

Documentacao tecnica do projeto

Gerado em 18/06/2026

Este documento explica como o projeto foi estruturado, quais tecnologias foram adicionadas e o papel de cada camada da solucao. A organizacao segue uma arquitetura em camadas, separando API, aplicacao, dominio, infraestrutura e testes.



Fluxo principal: a API recebe a requisicao, chama os casos de uso, aplica regras do dominio e delega persistencia/mensageria a infraestrutura.

1. Visao Geral

O DevFreela foi preparado como uma API em .NET 8 para gerenciar projetos de freelancers. A solucao passou a ter uma borda HTTP em ASP.NET Core, regras de aplicacao em servicos, contratos e entidades no Core, persistencia com Entity Framework Core, mensageria com RabbitMQ, testes com xUnit e artefatos de containerizacao e Azure.

Item	Descrição
Solucao	Criado o arquivo DevFreela.sln com API, Application, Core, Infrastructure e UnitTests.
API	Criado projeto ASP.NET Core com controllers, JWT, health check e configuracoes.
Persistencia	DevFreelaDbContext passou a usar DbSet e mapeamentos com OnModelCreating.
Mensageria	Adicionada abstracao IMessageBus e implementacao RabbitMQ para evento project-created.
Testes	Criado projeto xUnit com Moq para validar ProjectService.
Operacao	Adicionados Dockerfile, docker-compose.yml, .dockerignore, .gitignore e Bicep para Azure.

Comandos principais

```
dotnet build DevFreela.sln
dotnet test DevFreela.sln
docker compose up --build
```

2. Estrutura Da Solucao

Projeto/Pasta	Responsabilidade
DevFreela.API	Entrada HTTP da aplicacao. Contem Program.cs, controllers, modelos de login, configuracao JWT e appsettings.
DevFreela.Application	Camada de casos de uso. Contem servicos, interfaces de servicos, InputModels, ViewModels e DI da aplicacao.
DevFreela.Core	Camada de dominio e contratos. Contem entidades, enums, interfaces de repositorio, evento de integracao e contrato de mensageria.
DevFreela.Infrastructure	Implementacoes externas. Contem EF Core, repositorios concretos, configuracao de DI e publicador RabbitMQ.
DevFreela.UnitTests	Testes unitarios com xUnit e Moq.
infra/azure	Infraestrutura como codigo para Azure Container Apps e Azure SQL.

3. Camada API

A camada DevFreela.API é a porta de entrada do sistema. Ela registra controllers, injeta as camadas Application e Infrastructure, configura autenticação JWT e expõe um endpoint de saúde em /health.

- Program.cs: monta o pipeline HTTP com UseAuthentication, UseAuthorization e MapControllers.
- AuthController: gera token JWT a partir de email, senha e role. Em desenvolvimento usa a senha demo configurada em Jwt:DemoPassword.
- ProjectsController: expõe endpoints de consulta, criação, atualização, exclusão, comentários, início e finalização de projetos.
- SkillsController: expõe consulta de skills.
- appsettings.json: centraliza connection string, JWT, RabbitMQ e flag Database:EnsureCreated.

Endpoints principais

Método	URL	Autenticação	Descrição
POST	/api/auth/login	Anônimo	Gerar accessToken JWT.
GET	/api/projects	Anônimo	Listar projetos, com filtro opcional query.
GET	/api/projects/{id}	Anônimo	Consultar detalhes de um projeto.
POST	/api/projects	Admin ou Client	Criar projeto e publicar evento project-created.
PUT	/api/projects/{id}/start	Admin ou Freelancer	Iniciar projeto.
PUT	/api/projects/{id}/finish	Admin ou Freelancer	Finalizar projeto.
GET	/api/skills	Anônimo	Listar skills cadastradas.

4. Camada Application

A camada Application contém os casos de uso. Ela não conhece EF Core nem RabbitMQ diretamente; trabalha com contratos do Core, como IProjectRepository, ISkillRepository e IMessageBus. Isso deixa a regra de aplicação mais fácil de testar.

- ProjectService: cria, lista, atualiza, remove, inicia, finaliza e comenta projetos.
- SkillService: lista skills usando ISkillRepository.
- InputModels: representam dados recebidos pela API, como NewProjectInputModel e UpdateProjectInputModel.
- ViewModels: representam dados devolvidos pela API, como ProjectViewModel e ProjectDetailViewModel.
- DependencyInjectionExtensions: registra IProjectService e ISkillService.

O ProjectService publica ProjectCreatedIntegrationEvent na fila project-created logo depois que o repositório confirma a criação do projeto.

5. Camada Core

A camada Core guarda o modelo de domínio e os contratos independentes de tecnologia. Ela é a parte mais estável da aplicação, pois não depende de ASP.NET Core, EF Core ou RabbitMQ.

Componente	Responsabilidades
Project	Entidade principal. Controla titulo, descricao, cliente, freelancer, custo, status e transicoes Start, Finish, Cancel e Update.
User, Skill, UserSkill	Representam usuarios, competencias e relacionamento entre usuario e skill.
ProjectComment	Representa comentarios associados a projetos.
ProjectStatusEnum	Define estados do projeto, como Created, InProgress, Cancelled e Finished.
IProjectRepository / ISkillRepository	Contratos de persistencia usados pela Application.
IMessageBus	Contrato de publicacao de mensagens.
ProjectCreatedIntegrationEvent	Evento publicado quando um projeto e criado.

6. Camada Infrastructure

A camada Infrastructure implementa detalhes externos: banco de dados SQL Server via Entity Framework Core e mensageria via RabbitMQ. Ela registra essas implementacoes no container de injecao de dependencia.

Entity Framework Core

- DevFreelaDbContext passou a expor DbSet para Projects, Users, Skills, Comments e UserSkills.
- OnModelCreating configura chaves, tamanho de campos, precisao decimal, relacionamento de comentarios e dados iniciais.
- ProjectRepository usa metodos async e SaveChangesAsync para persistir alteracoes.
- SkillRepository consulta skills com AsNoTracking para leitura eficiente.

RabbitMQ

- RabbitMqOptions concentra HostName, UserName, Password e QueueName.
- RabbitMqMessageBus serializa a mensagem em JSON, declara a fila e publica no routing key informado.
- A fila local padrao e project-created.

7. Fluxo De Criacao De Projeto



Fluxo principal: a API recebe a requisicao, chama os casos de uso, aplica regras do dominio e delega persistencia/mensageria a infraestrutura.

- O cliente chama POST /api/projects enviando um Bearer token com role Admin ou Client.

- ProjectsController recebe NewProjectInputModel e chama IProjectService.CreateAsync.
- ProjectService cria a entidade Project usando regras do dominio.
- IProjectRepository.CreateAsync persiste no SQL Server via EF Core.
- Depois da persistencia, ProjectService publica ProjectCreatedIntegrationEvent em RabbitMQ.
- A API retorna HTTP 201 com referencia para consultar o projeto criado.

8. Autenticacao E Autorizacao JWT

A autenticacao foi implementada com Microsoft.AspNetCore.Authentication.JwtBearer. O token e assinado com a chave configurada em Jwt:SecretKey e validado pelo middleware antes dos endpoints protegidos.

Exemplo local de login:

```
POST /api/auth/login
{"email":"cliente@devfreela.com","password":"devfreela123","role":"Client"}
```

Em producao, a chave JWT, senha demo e credenciais devem vir de variaveis de ambiente ou secrets do provedor de nuvem, nunca fixas no codigo.

9. Docker, RabbitMQ E Banco Local

O Dockerfile usa build em dois estagios: SDK do .NET para restore/publish e runtime ASP.NET Core para executar a API. O docker-compose.yml sobe tres servicos: devfreela.api, sqlserver e rabbitmq.

Servico	Porta	Finalidade
devfreela.api	8080	API ASP.NET Core.
sqlserver	1433	Banco SQL Server usado pelo EF Core.
rabbitmq	5672 / 15672	Mensageria e painel de gerenciamento.

10. Azure

A pasta infra/azure contem um template Bicep para criar os recursos basicos em nuvem: Log Analytics, Azure Container Apps, Azure SQL Server, banco SQL e secrets para JWT, connection string e RabbitMQ.

- containerImage indica a imagem Docker publicada em um registry.
- jwtSecret e sqlAdminPassword sao parametros seguros.
- ConnectionStrings__DevFreelaCs e Jwt__SecretKey sao injetados como variaveis de ambiente no container.
- O output apiUrl retorna a URL publica da API apos deploy.

11. Testes

Foi criado o projeto DevFreela.UnitTests com xUnit e Moq. Os testes atuais focam ProjectService, validando que a criação chama o repositório e publica o evento de integração, além de verificar retorno nulo quando um projeto não existe.

Teste	Descrição
CreateAsync publica evento	O projeto é criado, o id é retornado e o evento project-created é publicado.
GetByIdAsync retorna null	O serviço retorna null quando o repositório não encontra o projeto.

Resultado validado

```
dotnet build DevFreela.sln: sucesso, 0 warnings, 0 erros.  
dotnet test DevFreela.sln: 2 testes aprovados.
```

12. Decisões Técnicas E Melhorias Feitas

- A camada Application deixou de depender de Infrastructure e passou a depender apenas de contratos do Core.
- Serviços e repositórios foram ajustados para padrão async/await.
- O DbContext foi corrigido para DbSet em vez de List em memória.
- Foram adicionados construtores privados nas entidades para compatibilidade com EF Core.
- Datas de criação e transição passaram a usar DateTime.UtcNow.
- O seed com data inválida foi corrigido.
- Foi criada .gitignore para evitar versionamento de bin, obj e arquivos locais do Visual Studio.
- O README foi atualizado com instruções de execução, login e testes.

13. Proximos Passos Recomendados

- Adicionar migrations EF Core em vez de depender de EnsureCreated.
- Criar autenticação real de usuários com hash de senha e tabela de credenciais.
- Adicionar consumers RabbitMQ para processar eventos de projeto criado.
- Adicionar testes de integração para controllers e persistência.
- Trocar secrets locais por Azure Key Vault ou secrets da plataforma.
- Configurar CI/CD para build, testes, publicação da imagem Docker e deploy Azure.